```
    return torch.clamp(x + mask,0,1)

def prepare_images(xt,maxtrans=6,rot=5,noise=10):
4
    out = torch.zeros(xt.shape)
    for i in range(xt.shape[0]):
        img = xt[i].unsqueeze(dim=0)
        img = TV.transforms.functional.to_pil_image(img)
        rand_rot = np.random.randint(-1*rot,rot,1) if rot > 0 else
0
        xtrans,ytrans = np.random.randint(-maxtrans,maxtrans,2)
        img = TV.transforms.functional.affine(img, rand_rot,
(xtrans,ytrans),1,0)
        img = TV.transforms.functional.to_tensor(img).squeeze()
        if noise > 0:
            img = add_spots(img,m=noise)
        maxval = img.view(-1).max()
        if maxval > 0:
            img = img.float() / maxval
        else:
            img = img.float()
        out[i] = img
    return out
```

- ***1*** Downloads and loads the MNIST training data
- ***2*** Downloads and loads the MNIST testing data for validation
- ***3*** Adds random spots to the image
- ***4*** Preprocesses the images and perform random transformations of rotation and translation

The `add_spots` function takes an image and adds random noise to it. This function is used by the `prepare_images` function, which normalizes the image pixels between 0 and 1 and performs random minor transformations such as adding noise, translating (shifting) the image, and rotating the image.

Figure 10.14 shows an example of an original and perturbed MNIST digit. You can see that the image is translated up and to the right and has random dots sprinkled in. This makes the learning task more difficult because our model must learn translational, noise, and rotational invariant features in order to successfully classify. The `prepare_images` function has parameters that let you tune how much the image will be perturbed, so you can control the difficulty of the problem.